



AI61002: Deep Learning Foundations and Applications

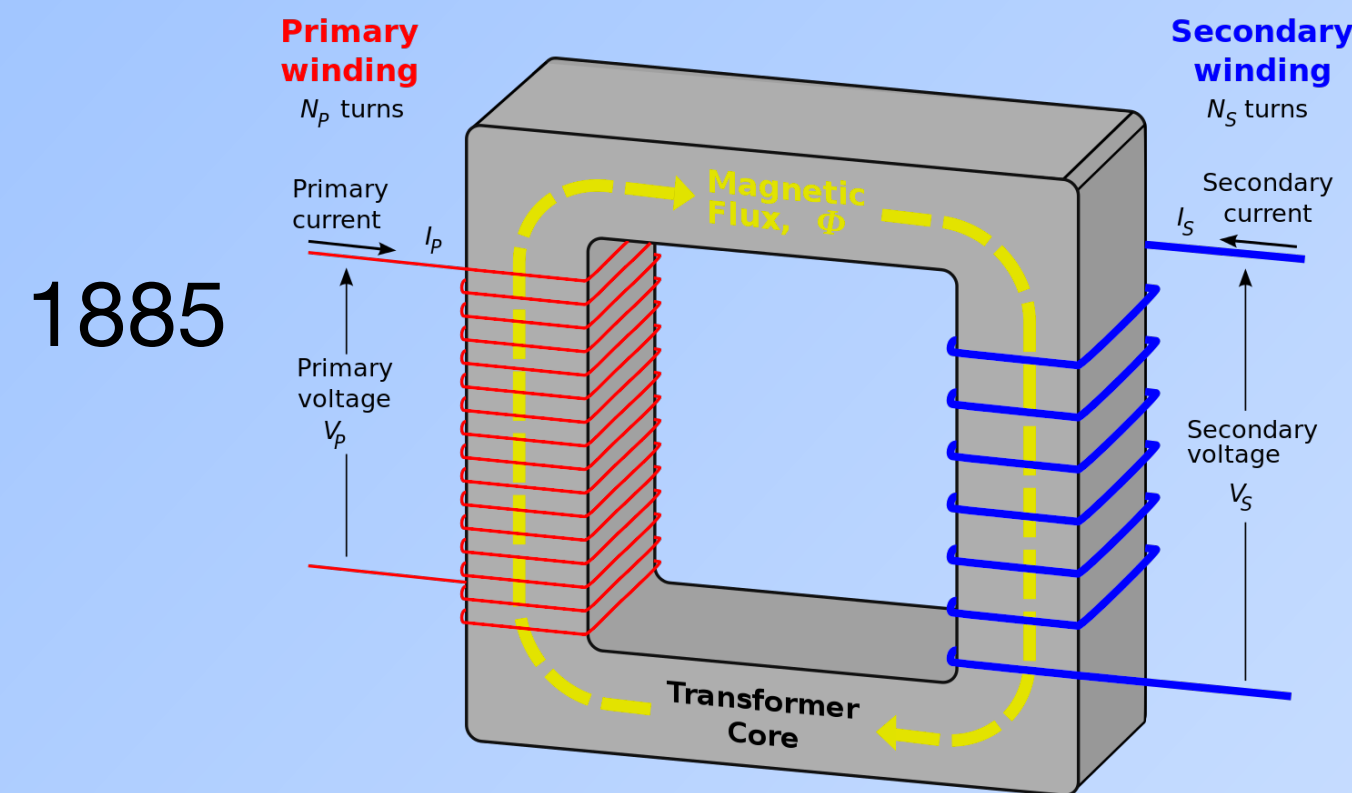
Module 7: Transformers

Dr. Somdyuti Paul

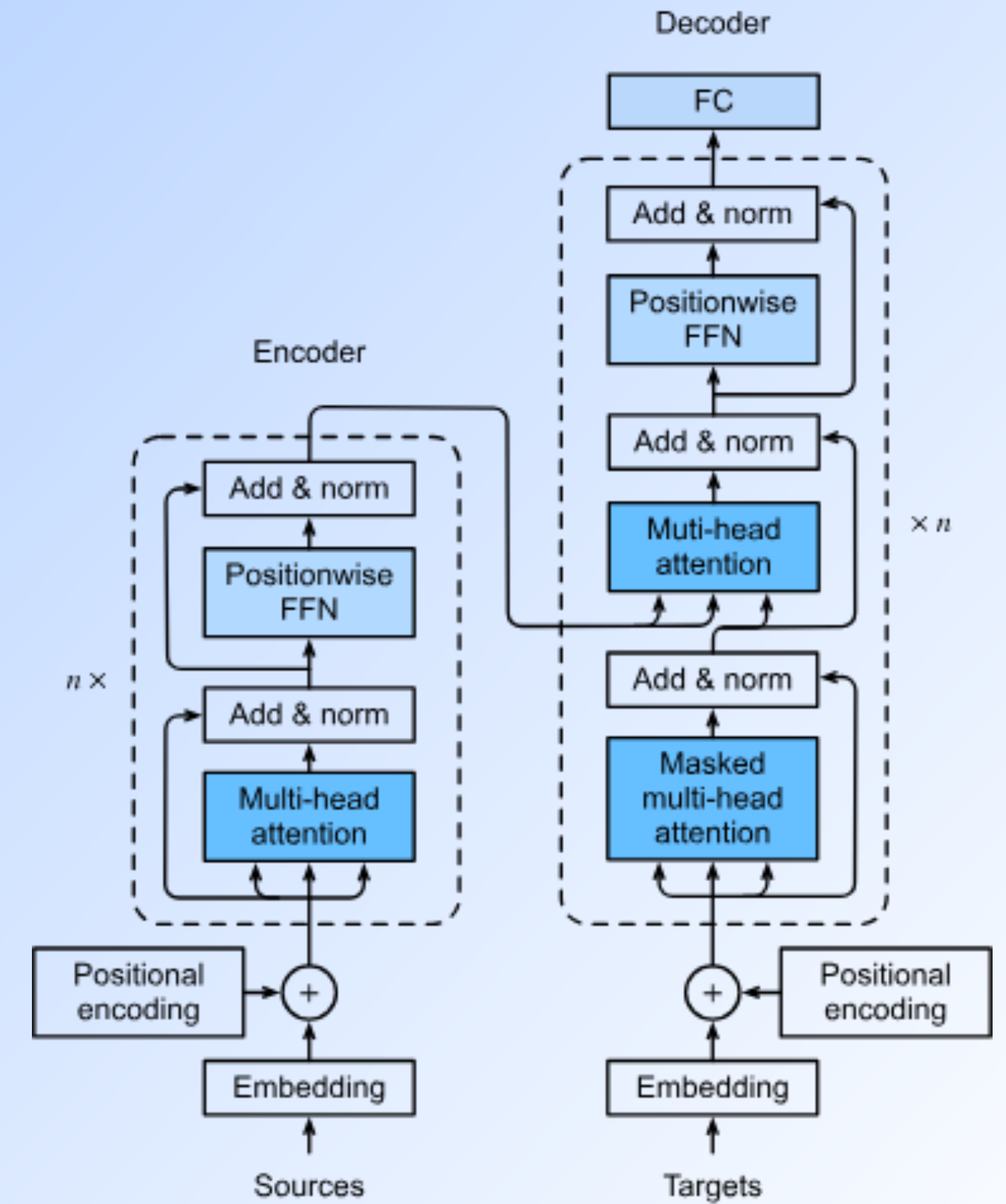
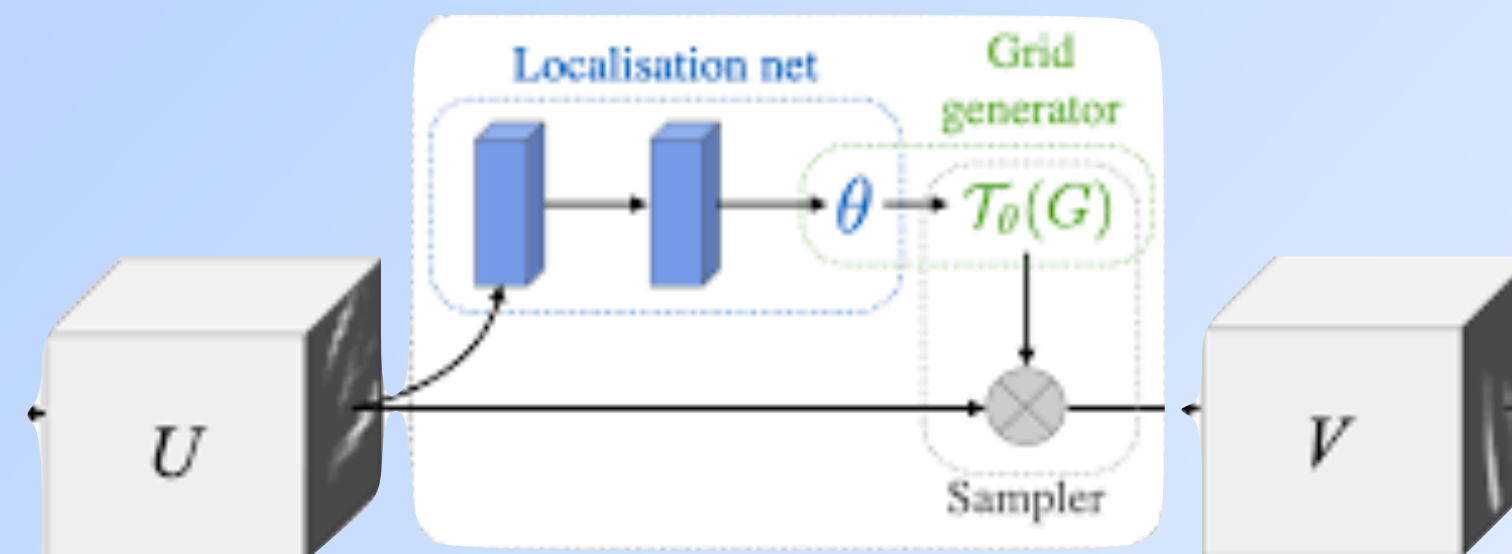
What Are Transformers?



1984



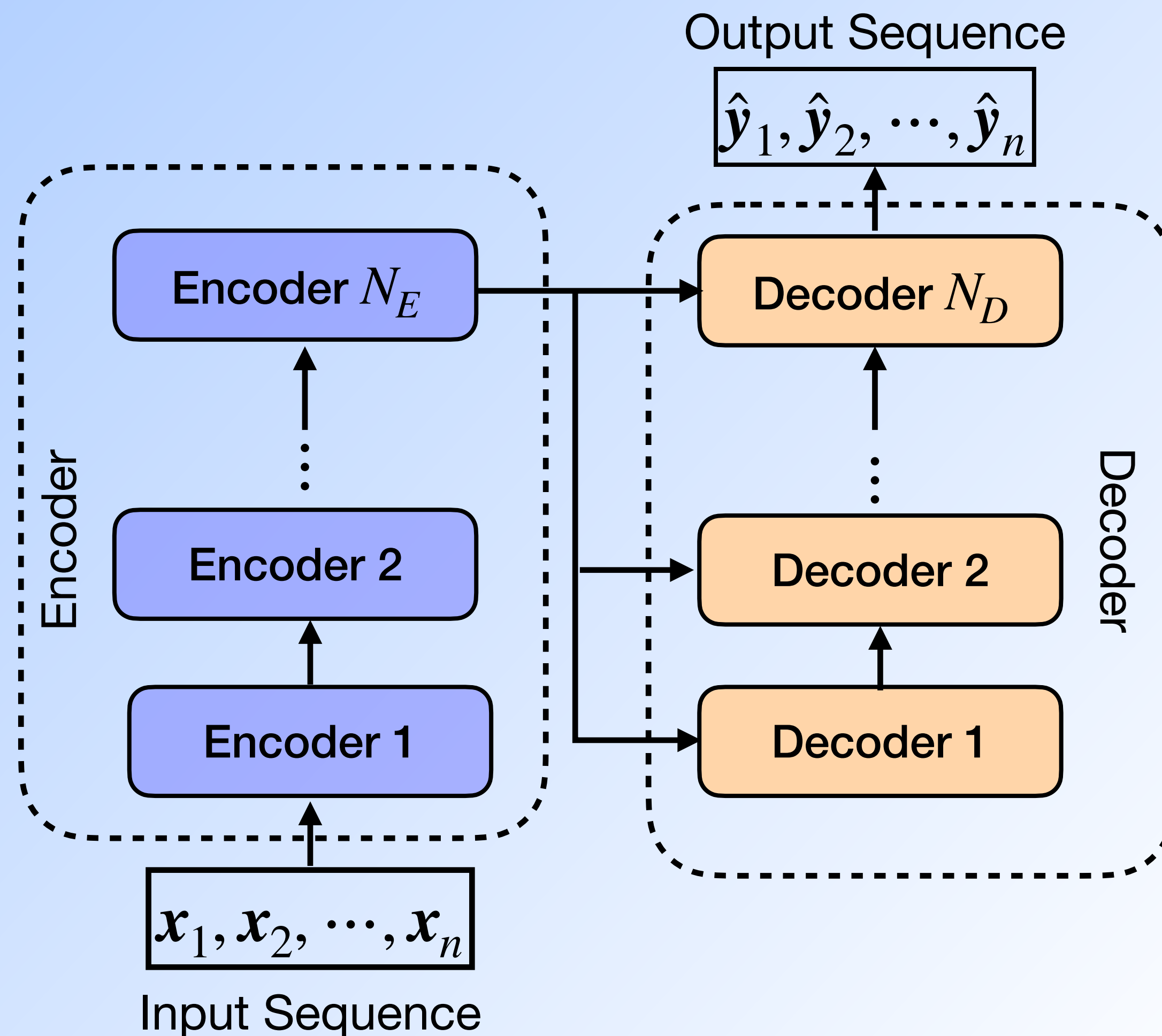
2015



2017

Transformers for Sequence Modeling

- Transformers are a class of deep neural networks that have achieved state-of-the-art performance on sequence modeling tasks since 2017 ([Vaswani et al., 2017](#)).
- Key features of transformer models are:
 - Stacked encoder decoder architecture
 - Attention mechanism
 - Parallelization



Input as Sequence of Tokens

- We will refer to the smallest unit of the input sequence that the transformer processes as a “token” (a term derived from NLP).
- An input sequence can be “tokenized” in multiple ways:
 - For text sequences: tokens can be chosen as words, sub-words or characters
 - For images: tokens can be chosen to be patches of arbitrary sizes.
 - For videos: tokens can be chosen to be frames or a group of frames of arbitrary length.
- For text data, tokens are mapped to their corresponding IDs using a predefined vocabulary to convert the data in numerical form.
- In addition to the tokens contained in the input sequence, special tokens are often used for classification, padding, masking, separator, unknown values, start and end of sequence, etc.

Tokens

40

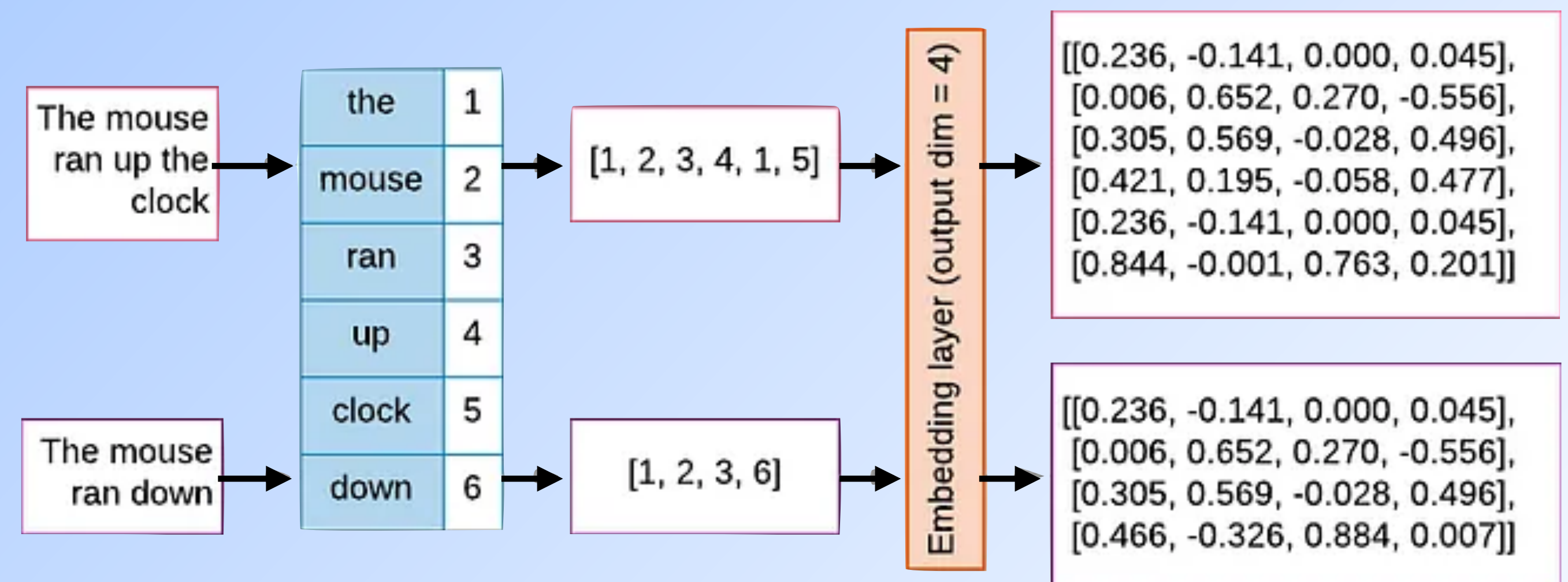
Characters

204

Specifically, tokens are the segments of text that are fed into and generated by the machine learning model. These can be individual characters, whole words, parts of words, or even larger chunks of text.

Token Embeddings

- The token IDs are mapped to vectors by using a learned projection/embedding matrix.
- Token embeddings help to capture contextual meaning of tokens, and their relationships in the input sequence.
- The model learns to map tokens with similar contextual meaning to vectors that are close to one another in the high dimensional space.



Positional Encoding

- Unlike RNNs, transformers do not process the input tokens sequentially, and hence lack an in-built mechanism of capturing order of tokens in the sequence.
- Positional encoding: positional encoding is added to the input embeddings to add information about the position of the tokens in sequence.
- Sinusoidal functions were employed to add positional encodings as follows:

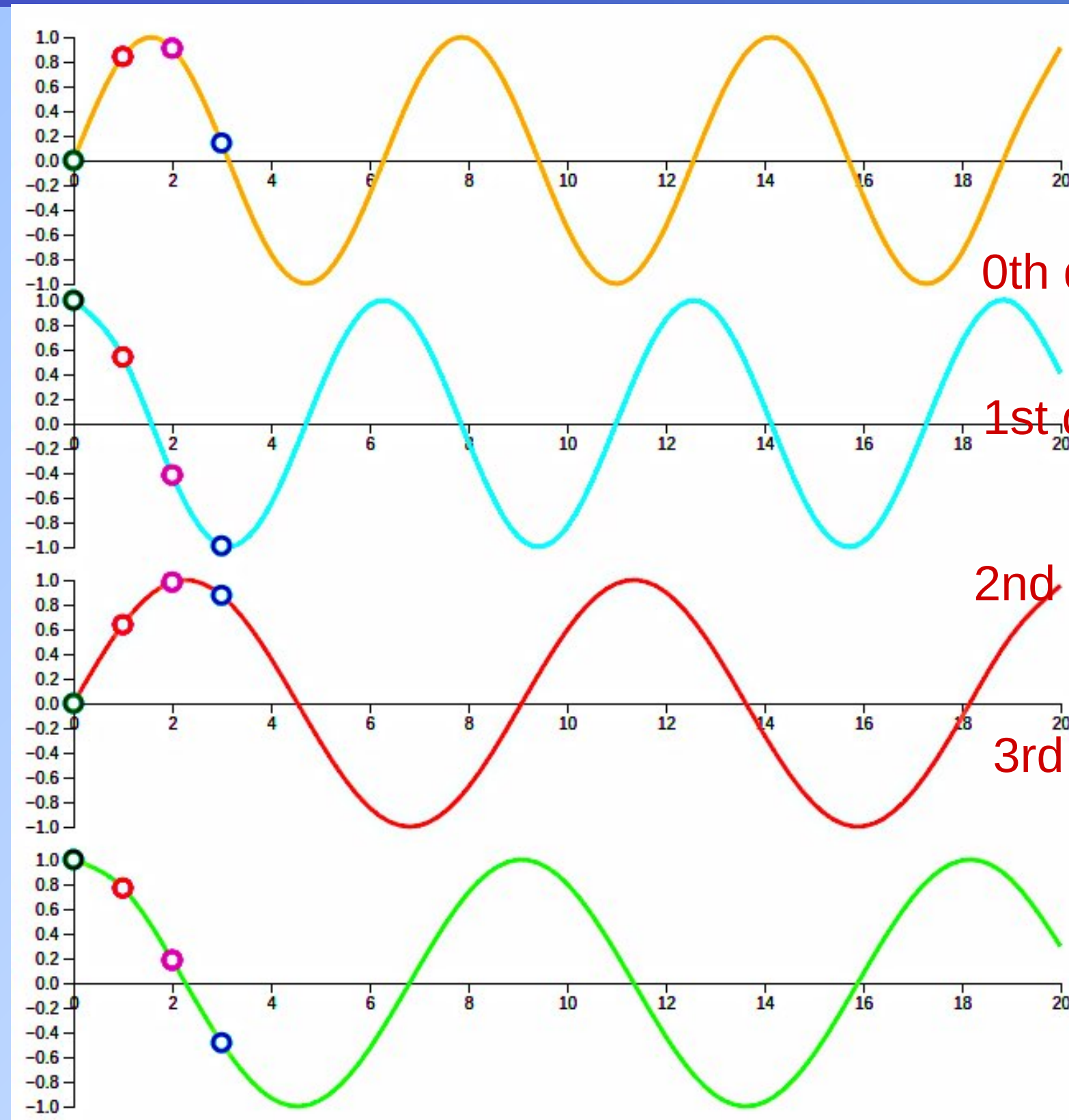
$$PE(i, 2k) = \sin \left(\frac{i}{10000^{\frac{2k}{d}}} \right), PE(i, 2k + 1) = \cos \left(\frac{i}{10000^{\frac{2k}{d}}} \right), \text{ where } i \text{ is the token position, } k \text{ denotes}$$

the index for the embedding dimension, and d is the dimensionality of the embeddings.

- The wavelengths of the above sinusoidal functions form a geometric progression.

Positional Encoding

- Thus, a unique encoding is generated for each position in the sequence up to twice the period of the sinusoid corresponding to the last dimension of the sequence.
- Positional encoding can also be learned.



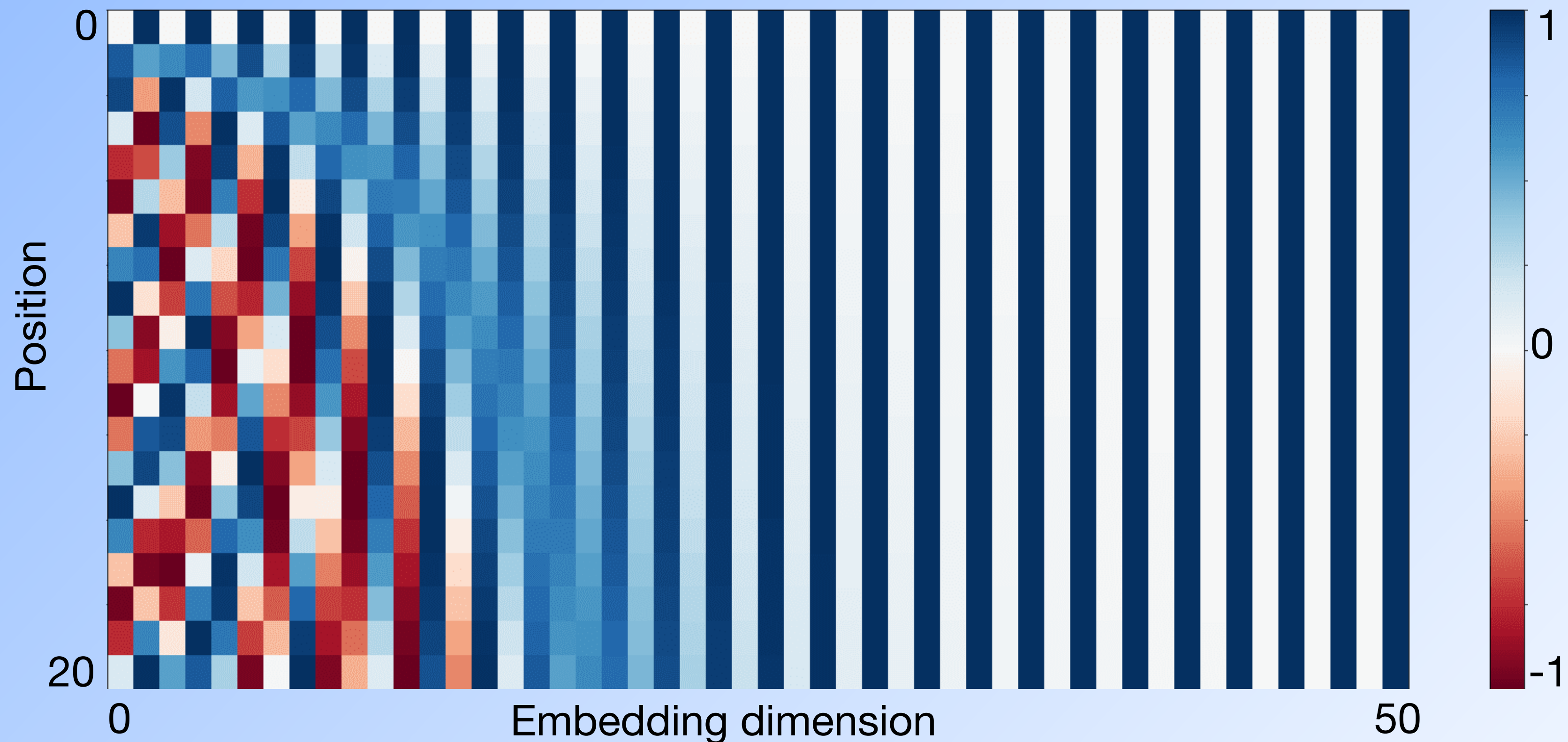
	0th token	1st token	2nd token	3rd token
0th dim	0.000	0.841	0.909	0.141
1st dim	1.000	0.540	-0.416	-0.990
2nd dim	0.000	0.638	0.983	0.875
3rd dim	1.000	0.770	0.186	-0.484

We are extracting code for each position. The 1st column is positional encoding for token at 0th position of the sequence. 2nd column is the positional encoding of the token at 1st position of the sequence.

Along Horizontal dimension we have variation of position and along column dimension we have variation of dimension of vector embedding. 'd' is increasing in downward direction, 'i' is increasing in horizontal direction. Each sinusoid is dedicated for a particular dim of diff tokens

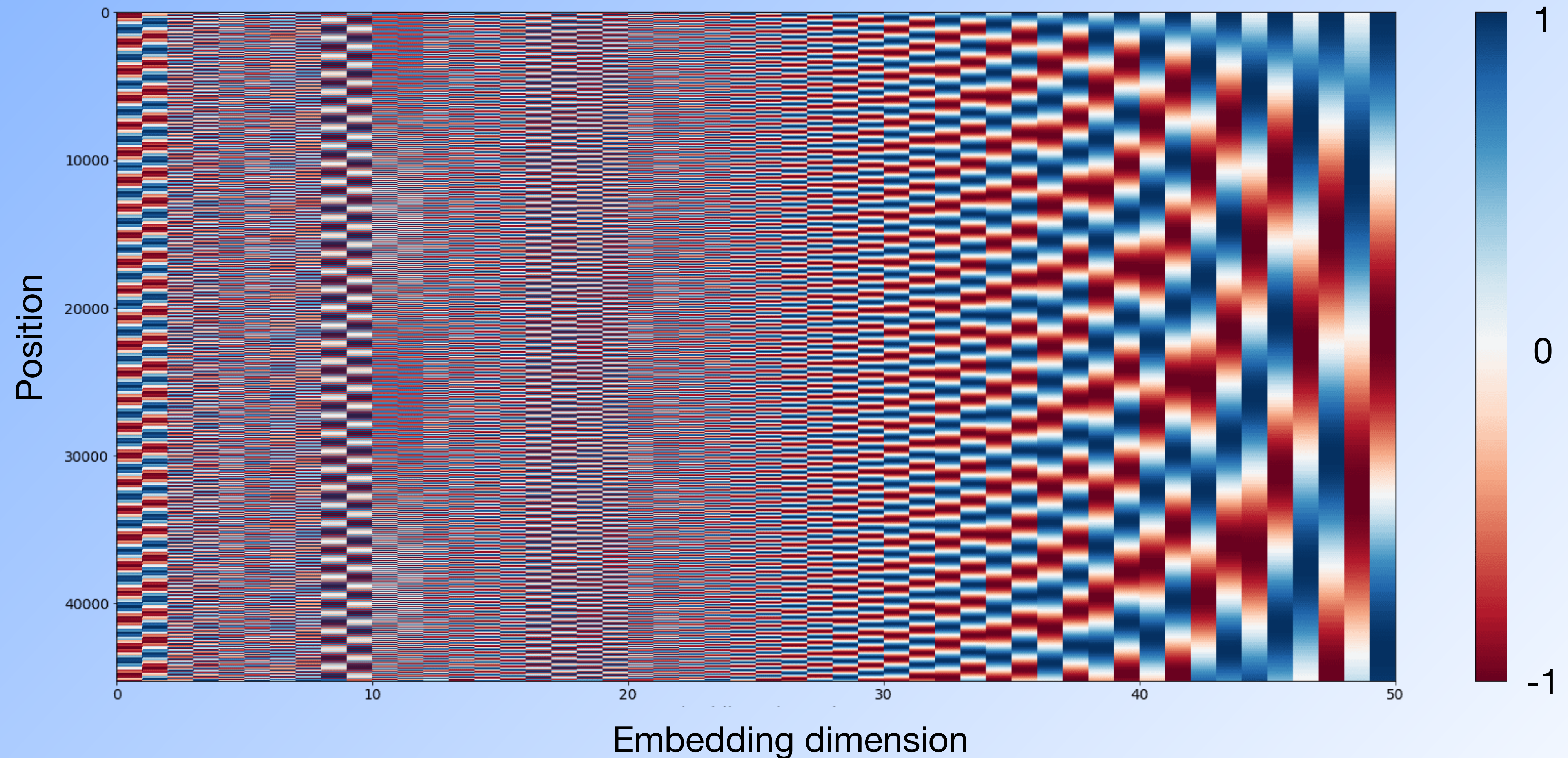
Positional Encoding

Visualizing positional encoding for first 20 positions:



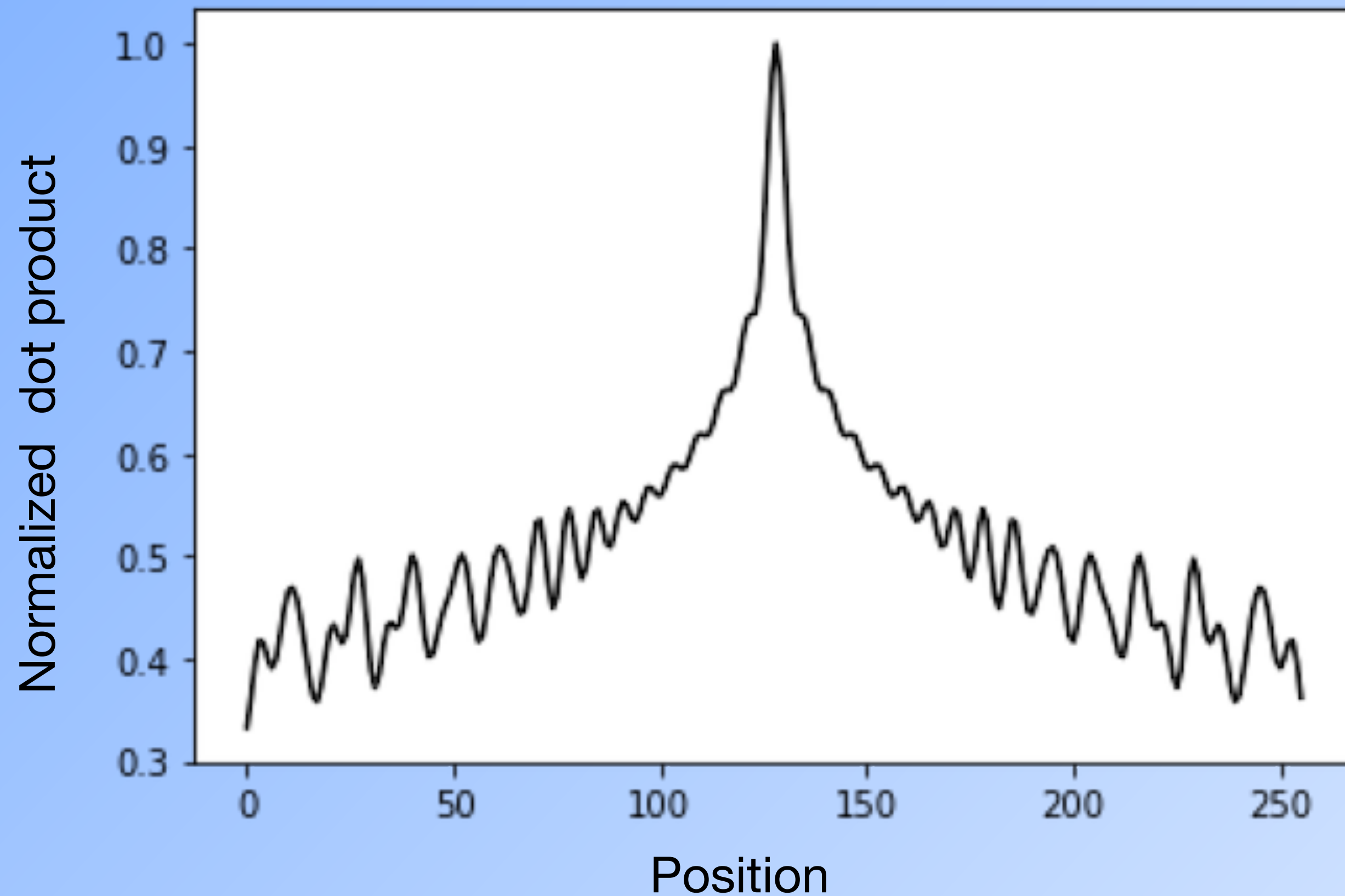
at $k = 0$, the variations are large due to large value of frequency, at $k = 50$ frequency is very small.
one row $\Rightarrow$ positional encoding corresponding to i _th token.

Positional Encoding

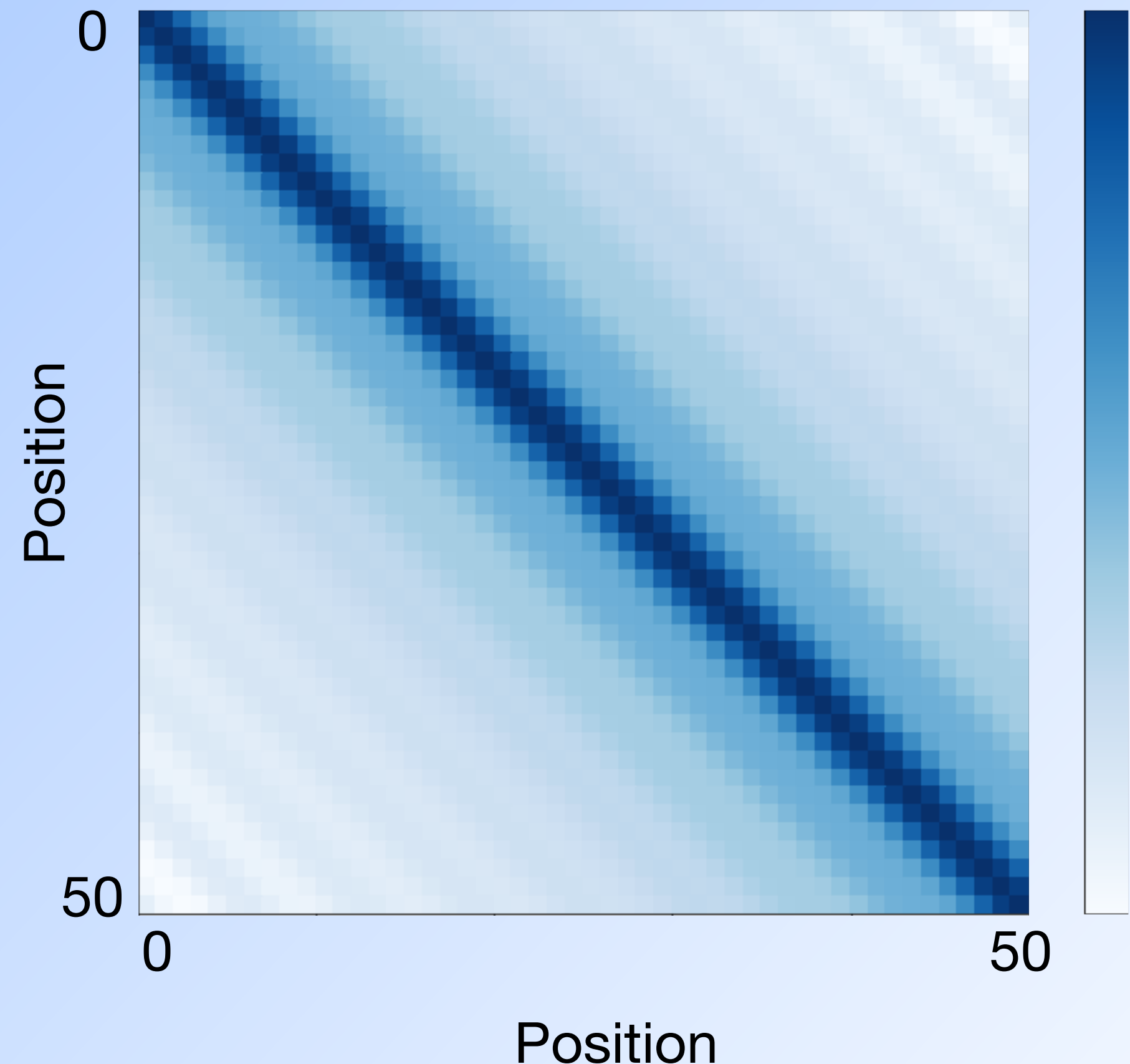


number of samples reqd by the nyquist rate than num of pixels there ⁹would be serious aliasing and we won't be getting correct visualizations

Positional Encoding



Dot product of position vector for position 128
with all other position vectors

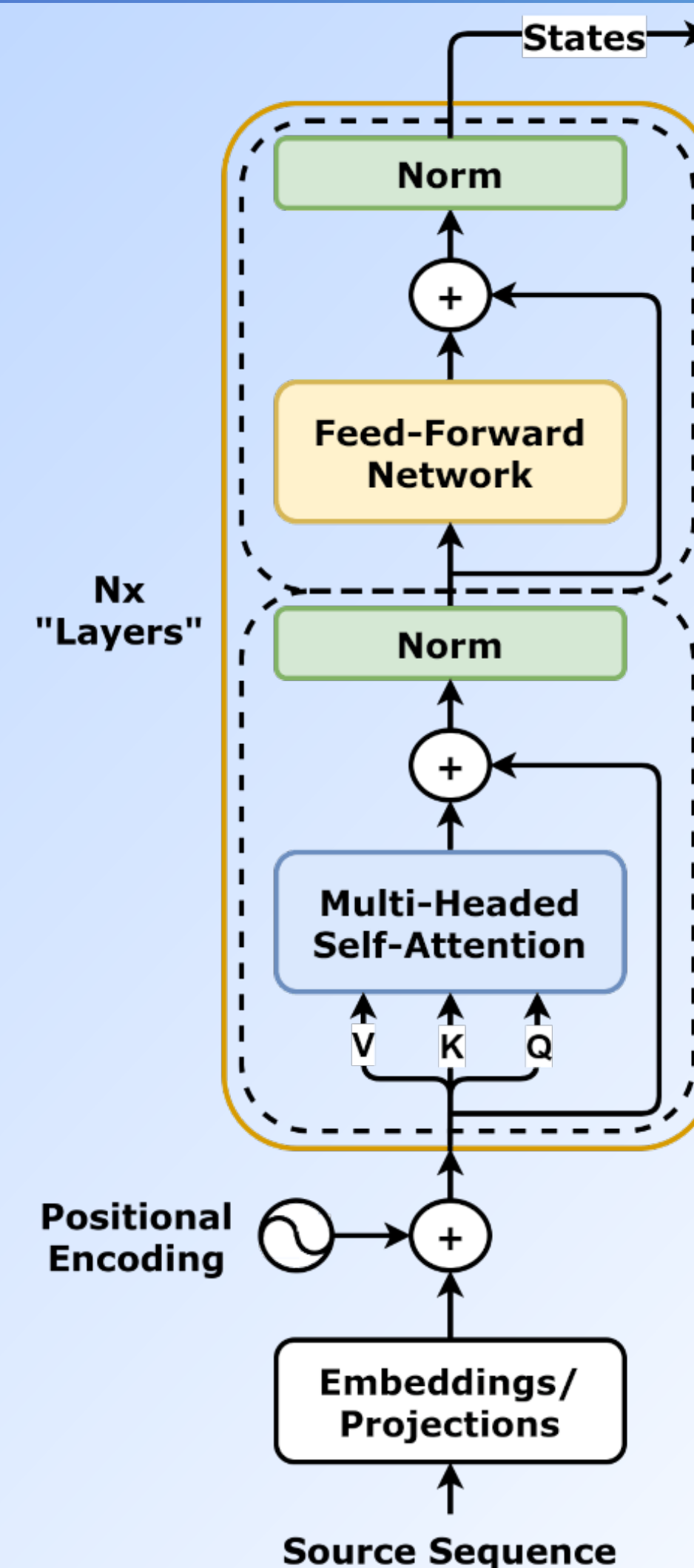


along diagonals we have : $PE(i)^T \text{ dot_product_with } PE(i)$
along off-diagonals we have : $PE(i)^T \text{ dot_product_with } PE(j), i \neq j$

we want dot product of vectors to be higher for positions nearby and
dot product to be lower for distant positions, coz we wanted to capture which tokens
are closest to each other.

The Transformer Encoder

- The transformer encoder processes the token embeddings of the entire sequence concurrently to capture the contextual relationship between the tokens.
- The input representation consisting of the token embeddings added with the positional encoding information, is passed through N encoder layers, where each encoder layer consists of the following sub-layers:
 - Multi-Headed Attention Sublayer
 - Positionwise Feedforward Neural Network Sublayer



Encoder - Self Attention Mechanism

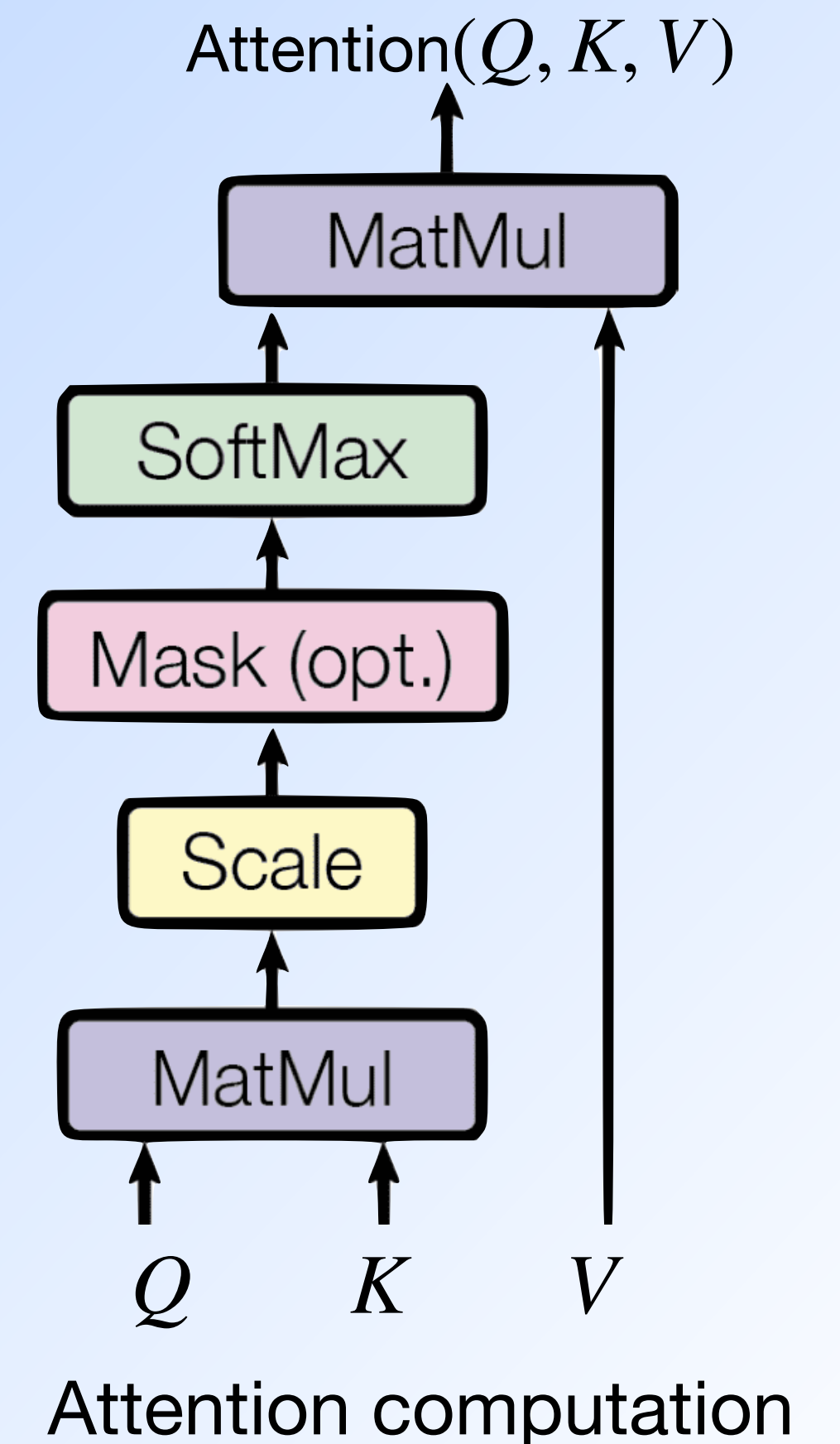
- The self-attention mechanism allows the transformer to relate every input token with every other token in the sequence, allowing the tokens to interact with each other.
- Let X be the input to the encoder layer having shape (n, d) , where n is the number of tokens in the sequence and d is the dimensionality of the token embedding.
- Three different learned linear projections are obtained from each token of the input X as follows:
 $Q = XW_Q$, $K = XW_K$, $V = XW_V$, where Q and K are each of dimension $n \times d_k$, and V is of dimension $n \times d_v$.
- These three linear projections are interpreted as follows:
 - Query (Q): query is the representation of the token that the model is currently focusing on.
 - Key (K): key represents all the other tokens in the sequence, including the one being currently focused on.
 - Value (V): the keys for each token is associated with a corresponding value, which represents the contextual or semantic meaning of the token.

Encoder - Self Attention Mechanism

- Each query is matched with all the keys in the sequence using a scaled dot product attention as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- QK^T is an $n \times n$ attention score matrix, where each element represents the attention (i.e. relationship) between two tokens.
- The raw attention scores given by QK^T are scaled by the $\sqrt{d_k}$ to prevent excessively large values of the dot product when d_k is large.
- The attention scores are normalized to sum to 1 using the softmax function.
- The normalized attention scores are subsequently used to weight the values in V to obtain the final attention to extract the contextual significance of each token in relation to a particular token.
- Optionally, a mask can be introduced to make each token attend to a set of tokens in the sequence rather than all the tokens (for e.g. masking padding tokens).



Before applying softmax, we just put a mask on the attention score matrix i.e. A_dash

Encoder - Multi-Headed Self Attention

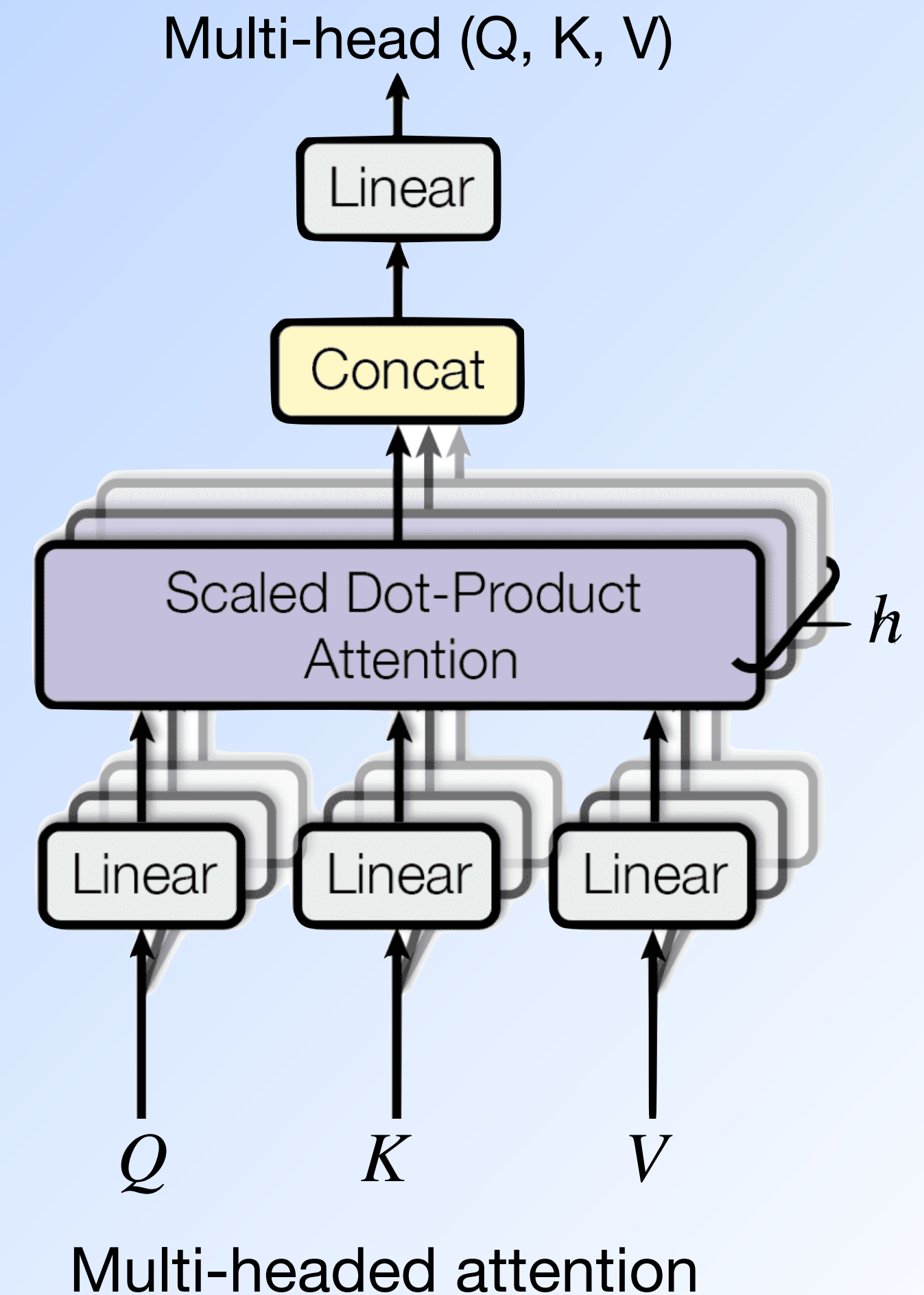
- Multi-Head Self Attention involves repeating the self-attention mechanisms multiple times, allowing the model to capture different aspects of the input sequence.
- Each attention head operates independently, with different learned weight matrices to generate different linear projections of the inputs Q , K and V :

Q , K and V :

$$Q_i = QW_i^Q, K_i = KW_i^K, V_i = VW_i^V, \text{ where } W_i^Q, W_i^K \in \mathbb{R}^{d \times d_k} \text{ and } W_i^V \in \mathbb{R}^{d \times d_v}$$

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{Multi-head (Q, K, V)} = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O, \text{ where } W^O \in \mathbb{R}^{hd_v \times d}.$$



Encoder - Normalization and Residual Connection

- The input of each sub-layer is added to its output using residual connections.
- The output after the residual connection is normalized using layer normalization (Ba et al., 2016) as follows:

$$\tilde{Z} = \text{LayerNorm}(Z), \text{ where } Z = X + \text{Multihead Attention}(Q, K, V)$$

- Layernorm normalizes activations across feature dimensions as follows:

$$\tilde{Z}_{ij} = \gamma_i \frac{Z_{ij} - \mu_i}{\sigma_i} + \beta_i, \text{ where } \mu_i = \frac{1}{d} \sum_{j=1}^d Z_{ij} \text{ and } \sigma_i^2 = \frac{1}{d} \sum_{j=1}^d (Z_{ij} - \mu_i)^2$$

- Layer normalization is preferred over batch normalization to scale the activations as sequences in a batch could be of different lengths and padded, making the batch statistics somewhat unreliable.

